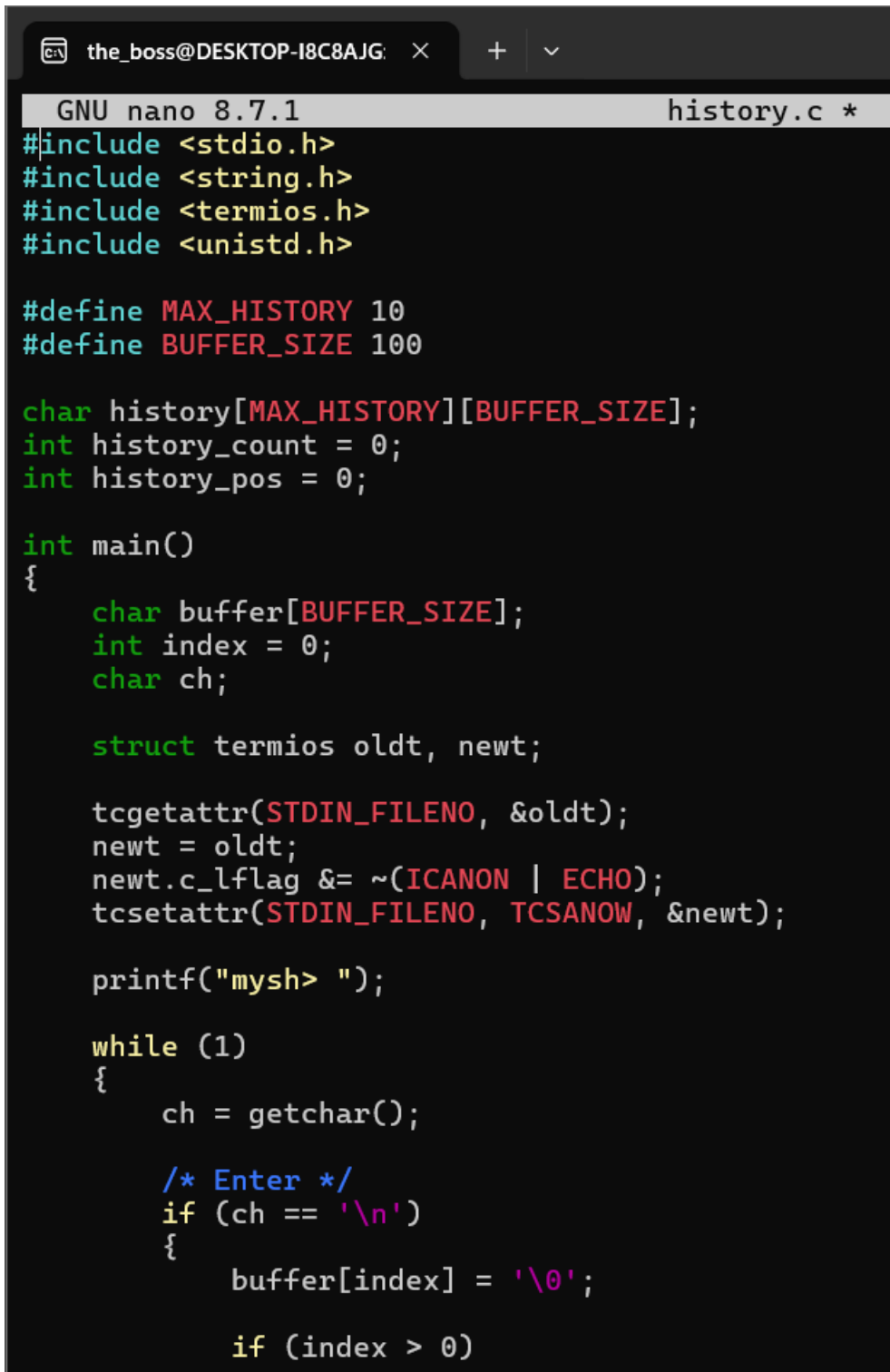


WEEK-2 SKILL

Session-3

Task 1

To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, and Test Recall Functionality.



```
the_boss@DESKTOP-I8C8AJG: × + v
GNU nano 8.7.1 history.c *
#include <stdio.h>
#include <string.h>
#include <termios.h>
#include <unistd.h>

#define MAX_HISTORY 10
#define BUFFER_SIZE 100

char history[MAX_HISTORY][BUFFER_SIZE];
int history_count = 0;
int history_pos = 0;

int main()
{
    char buffer[BUFFER_SIZE];
    int index = 0;
    char ch;

    struct termios oldt, newt;

    tcgetattr(STDIN_FILENO, &oldt);
    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    printf("mysh> ");

    while (1)
    {
        ch = getchar();

        /* Enter */
        if (ch == '\n')
        {
            buffer[index] = '\0';

            if (index > 0)
```

Output:

```
the_boss@DESKTOP-I8C8AJG:~/oss$ nano history.c
the_boss@DESKTOP-I8C8AJG:~/oss$ gcc history.c -o bin/history
the_boss@DESKTOP-I8C8AJG:~/oss$ ./bin/history
mysh> hello
You entered: hello
mysh> linux
You entered: linux
mysh> test
You entered: test
mysh> hello
You entered: hello
mysh> exit
You entered: exit

Exiting shell...
the_boss@DESKTOP-I8C8AJG:~/oss$ |
```

Report: I applied escape sequences to detect the Up and Down arrow keys and implemented command history to store previously entered commands. I tested navigating through previous and next commands and updated the input buffer accordingly. The command recall functionality was tested successfully, and the previously entered commands were displayed correctly.

Task-2

To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, and Verify with Valgrind.

```
the_boss@DESKTOP-I8C8AJG: × + v
GNU nano 8.7.1 memory.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node
{
    char *command;
    struct Node *next;
} Node;

int main()
{
    int size = 10;
    int count = 0;

    char *buffer = malloc(size * sizeof(char));

    if (buffer == NULL)
    {
        printf("Memory allocation failed\n");
        return 1;
    }

    printf("Enter a command: ");

    while (1)
    {
        char ch = getchar();

        if (ch == '\n')
            break;

        if (count >= size - 1)
        {
            size *= 2;

            char *temp = realloc(buffer, size * sizeof(char));
```

Output:

```
the_boss@DESKTOP-I8C8AJG:~/oss$ nano memory.c
the_boss@DESKTOP-I8C8AJG:~/oss$ gcc -g -Wall -Wextra memory.c -o bin/memory
the_boss@DESKTOP-I8C8AJG:~/oss$ ./bin/memory
Enter a command: hello world this is a dynamic memory test
Command: hello world this is a dynamic memory test
Stored command: hello world this is a dynamic memory test
Memory released successfully.
the_boss@DESKTOP-I8C8AJG:~/oss$ ~|
```

Report:

I implemented dynamic memory allocation using `malloc()` and resized the buffer using `realloc()` when more space was needed. I used a linked list structure to store the command and checked the allocated memory before using it. Proper `free()` functions were used to release the allocated memory. Finally, I tested the program with Valgrind and verified that there were no memory leaks.

Session-4

Task-1

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output

```
the_boss@DESKTOP-I8C8AJG: × + v
GNU nano 8.7.1 parser.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_TOKENS 50

typedef struct
{
    char *value;
} Token;

int main()
{
    char input[200];
    Token tokens[MAX_TOKENS];
    int count = 0;

    printf("Enter a command: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    char *token = strtok(input, " \t");

    while (token != NULL && count < MAX_TOKENS)
    {
        tokens[count].value = token;
        count++;

        token = strtok(NULL, " \t");
    }

    printf("\nTokens:\n");

    for (int i = 0; i < count; i++)
```

Output:

```
the_boss@DESKTOP-I8C8AJG:~/oss$ nano parser.c
the_boss@DESKTOP-I8C8AJG:~/oss$ gcc -Wall -Wextra parser.c -o bin/parser
the_boss@DESKTOP-I8C8AJG:~/oss$ ./bin/parser
Enter a command:  ls -l /home/user

Tokens:
Token 1: [ls]
Token 2: [-l]
Token 3: [/home/user]

Total tokens: 3
the_boss@DESKTOP-I8C8AJG:~/oss$ |
```

Report: I split the user input into individual tokens using delimiters such as spaces and tabs. I handled extra whitespace and stored each token using a token structure. The token stream was then displayed and checked to make sure the input was divided correctly. I tested the program with different commands and verified the parsing output.

Task-2

To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

```
the_boss@DESKTOP-I8C8AJG: × + ▾
GNU nano 8.7.1 parse_tree.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node
{
    char *value;
    struct Node *next;
} Node;

int main()
{
    char input[200];

    printf("mysh> ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
    {
        printf("Empty command\n");
        return 0;
    }

    char *token = strtok(input, " \t");

    if (token == NULL)
    {
        printf("Syntax error: empty command\n");
        return 1;
    }

    Node *head = NULL;
    Node *tail = NULL;

    while (token != NULL)
```

Output:

```
the_boss@DESKTOP-I8C8AJG:~/oss$ nano parse_tree.c
the_boss@DESKTOP-I8C8AJG:~/oss$ gcc -Wall -Wextra parse_tree.c -o bin/parse_tree
the_boss@DESKTOP-I8C8AJG:~/oss$ ./bin/parse_tree
mysh> ls -l /home

Parse Structure:
[ ls ] -> [ -l ] -> [ /home ]
Parsing completed successfully.
the_boss@DESKTOP-I8C8AJG:~/oss$ ./bin/parse_tree
mysh>
Empty command
the_boss@DESKTOP-I8C8AJG:~/oss$ |
```

Report:

I designed basic parser logic to process the tokens and create a simple parse structure using linked nodes. The program validates the input, detects empty commands, and displays syntax errors when required. The generated structure can be used as an execution structure for further shell processing. I tested the parser with different commands and verified the output successfully.